



Intelligent Forecasting of Appliance Usage Patterns: A Sequence-to-Sequence Model Approach

MSc Data Science and Machine Learning

Candidate Number: 19080357

Supervisor: Matthew Caldwell, Mica Menard, Boruo Xu

September 2023

Disclaim: This report is submitted as part requirement for the MSc Data Science and Machine Learning at University College London. It is substantially the result of my own work except where explicitly indicated in the text.

The report will be distributed to the internal and external examiners, but thereafter may not be copied or distributed except with permission from the author.

Abstract

As smart home technology advances, there's an increasing need for intelligent systems to predict household appliance usage patterns accurately. Traditional time series forecasting models, however, struggle with the sporadic and intermittent demand patterns of many appliances, leading to unsatisfactory predictions. To address this, we introduce a sequence-to-sequence model that transforms the dataset into discrete sequences of activation events, with each event capturing an appliance's ID and its activation time. By utilizing a sliding window methodology informed by the past M activation events, our model forecasts the subsequent N activation events. This approach effectively handles the inherent variability in household appliance usage, leading to enhanced prediction accuracy. The potential applications of our method promise not only improved energy efficiency but also deeper insights into consumer behavior.

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Objectives	4
2	Background	6
2.1	Recurrent Model	6
2.2	Sequence Modelling	11
2.2.1	Token by Token Processing	11
2.2.2	Encoder-Decoder Architecture $\rightarrow$ Seq2Seq	11
2.2.3	Attention Mechanisms & Transformers	12
2.3	Training Issues in Seq2Seq Tasks	14
2.4	Metrics	15
3	Pretreatment	18
3.1	Dataset	18
3.1.1	Preprocessing	18
3.1.2	Model inputs and outputs	19
3.1.3	Train test split	20
4	Model development	22
4.1	LSTM seq2seq model	22
4.2	Benchmark	24
4.3	LSTM seq2seq model with multi-head attention	24
5	Experiments	25
5.1	Model compare	25
5.2	Hyper-Parameter Tuning	26
5.2.1	Batch size	26
5.2.2	learning rate	26
5.2.3	Example output	27

6	Conclusions	30
6.1	Summary	30
6.2	Drawbacks	31
6.3	Future work	32
7	reference	34

1 Introduction

In this chapter, we describe the motivation for the topic. We will then present the problem that the paper needs to address and describe the objectives of the project.

1.1 Motivation

In the age of interconnected devices and smarter homes, we stand on the precipice of a data-driven revolution that has the potential to redefine how households consume energy. Yet, as technology has advanced, traditional methods for predicting appliance usage have failed to keep pace, often proving inadequate in the face of the intermittent demand patterns typical of household appliances. There exists a compelling need for:

- **User Experience:** As smart homes evolve, so do user expectations. Accurate predictions can lead to automated systems that cater to user needs even before they articulate them, leading to a superior user experience.
- **Methodological Constraints:** The limitations of traditional time series forecasting models, which frequently yield inaccurate predictions due to their inability to handle intermittent demand, necessitate the exploration of advanced modeling techniques [1].

1.2 Objectives

Due to the intricate use of modern household appliances, many of them are activated with low frequency and duration, which results in a large number of zero values in the raw data, which poses a great challenge to the traditional continuous time series prediction. This intermittent demand disrupts the continuity and leads to increased prediction errors in traditional models. Therefore, our study sets forth with the following objectives:

- **Data Transformation:** convert the continuous data, marked by its intermittent nature, into discrete pattern sequences, providing a clearer and more tractable foundation for subsequent analyses.
- **Seq2Seq Modeling:** Apply sequence-to-sequence models, which have demonstrated promise in capturing sequential data patterns [2]
- **Model Evaluation:** Test the efficacy of our proposed model in tangible, real- world settings, benchmarking it against several models to demonstrate superiority.

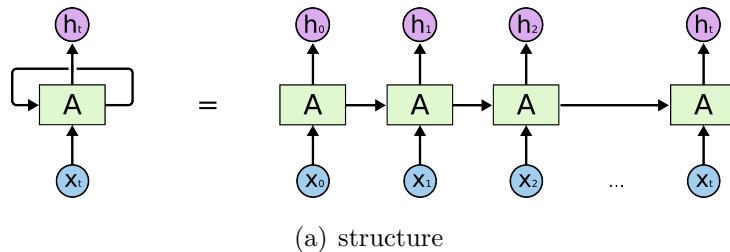
2 Background

2.1 Recurrent Model

Recurrent neural network

Recurrent Neural Networks (RNNs) represent a family of neural networks that are particularly suitable for processing sequences of data. They have internal loops to maintain information across time steps, allowing them to capture temporal dependencies [3].

The initial appeal of RNNs in sequential data processing can be attributed to their capacity to maintain and update their internal state as they process sequences. This ability has made RNNs a prime candidate for tasks ranging from time series forecasting to natural language processing.

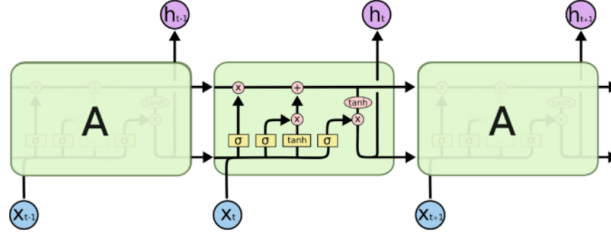


However, vanilla RNNs suffered from challenges related to training, notably the vanishing and exploding gradient problems [4]. These issues made it difficult for RNNs to capture long-term dependencies in data, limiting their effectiveness in some applications.

To overcome these challenges, variations of the RNN architecture were introduced. The Long Short-Term Memory (LSTM) [5] and the Gated Recurrent Unit (GRU) [6] are two notable advancements. They introduced gating mechanisms that allow for more effective training and better retention of long-term dependencies in sequences.

LSTM

Long Short-Term Memory (LSTM) units, introduced by Hochreiter and Schmidhuber in 1997 [5], are a specialized architecture within the broader family of Recurrent Neural Networks (RNNs). LSTMs were developed to tackle the vanishing and exploding gradient problems of traditional RNNs, which hampered them from effectively modeling long-term dependencies in sequence data.



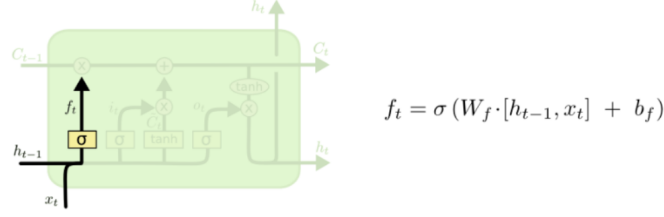
(b) structure

Structure and Formulation of LSTM Units

LSTM introduces several gating mechanisms to modulate the flow of information through the cell, enabling it to retain or forget information based on its relevance.

- **Forget Gate (f_t):** This gate decides the amount of information from the previous cell state C_{t-1} that should be kept or discarded. Formulated as:

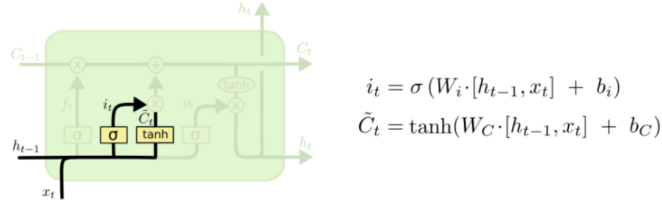
$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$



(c) forget gate

- **Input Gate (i_t):** Determines which parts of the new candidate values should be stored in the cell state. It's defined as:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$



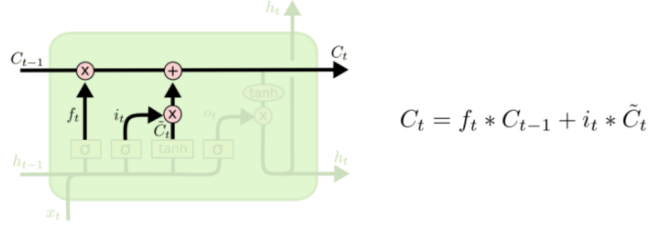
(d) input gate

- **Cell Candidate ($\tilde{C}_t$):** Represents the potential new information to be added to the cell state:

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

- **Cell State Update:** The cell state C_t at time t is updated based on the previous cell state modulated by the forget gate and the new candidate values modulated by the input gate:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$



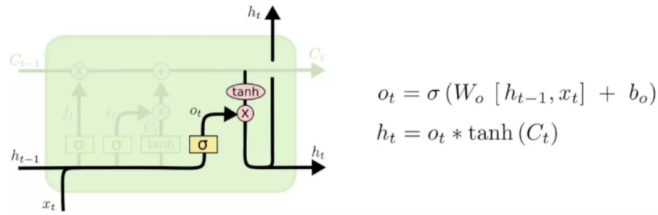
(e) output gate

- **Output Gate (o_t):** Decides how much information from the updated cell state will be output to the hidden state h_t and conveyed to the next LSTM cell or subsequent layers:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

The hidden state is then updated as:

$$h_t = o_t \odot \tanh(C_t)$$



(f) output gate

Where σ represents the sigmoid activation function and $\odot$ denotes element-wise multiplication.

GRU

Gated Recurrent Units (GRUs) were introduced by Cho et al. [6] as an alternative to the traditional Recurrent Neural Network (RNN) and the Long Short-Term Memory (LSTM). GRUs are designed to capture long-term dependencies while being computationally more efficient than LSTMs.

The GRU controls the flow of information like the LSTM but without having a memory unit. It uses two gates: a reset gate r and an update gate z . These gates determine how to combine new input with the previous memory.

The mathematical formulations for the GRU are as follows:

- Update gate:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad (1)$$

- Reset gate:

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad (2)$$

- New memory:

$$\tilde{h}_t = \tanh(W x_t + r_t \odot (U h_{t-1}) + b) \quad (3)$$

- Final memory:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (4)$$

Here, x_t is the input at time t , h_{t-1} is the previous hidden state, σ is the sigmoid activation function, and $\odot$ denotes element-wise multiplication. W, U and b are learnable parameters of the model.

The gates in the GRU help decide what information to pass onto the output. The reset gate helps the model determine how much of the past information to forget, while the update gate defines how much of the new information to store.

2.2 Sequence Modelling

Sequence modelling pertains to the techniques used to predict or classify sequences of data points. The key idea is to utilize the inherent order of the data, often temporally arranged, to derive meaningful patterns and relationships. This is especially relevant in fields like natural language processing, time series forecasting, and any domain where the order and context of data points are crucial [7].

2.2.1 Token by Token Processing

Token-by-token processing involves handling sequences one element at a time. In the context of natural language processing, a token typically refers to a word (or subword). Sequential models, especially Recurrent Neural Networks (RNNs) and their variants like Long Short-Term Memory networks (LSTMs) and Gated Recurrent Units (GRUs), process input sequences token-by-token. This approach helps the model to consider the past context when processing the current token, thereby capturing the dependencies and relationships across the sequence.

2.2.2 Encoder-Decoder Architecture $\rightarrow$ Seq2Seq

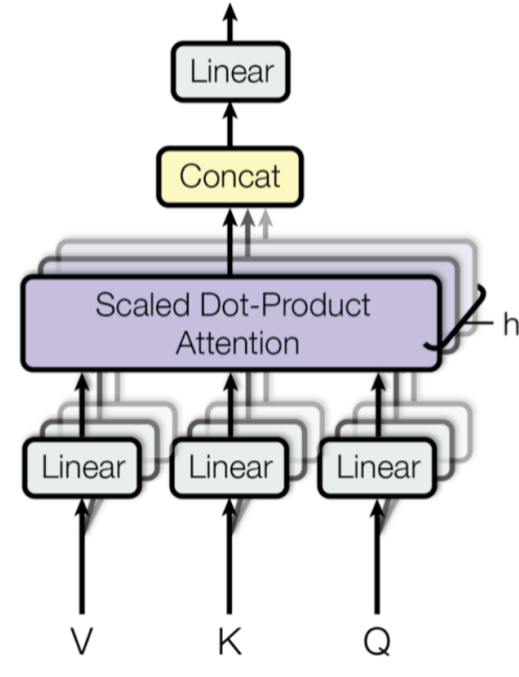
The Encoder-Decoder architecture, often termed as Seq2Seq, is pivotal for tasks that involve translating sequences into other sequences. The architecture comprises two primary components:

- **Encoder:** This captures the context of the input sequence and compresses it into a fixed-size context vector, often the final hidden state of the encoder RNN.
- **Decoder:** It utilizes the context vector from the encoder to produce the output sequence. The decoder often has its own RNN which generates the output sequence token-by-token, conditioned on the context vector and the previously generated tokens.

This architecture has been widely used in tasks such as machine translation [2].

2.2.3 Attention Mechanisms & Transformers

The attention mechanism was introduced to overcome the limitations of the encoder-decoder architecture, mainly the challenge of compressing long sequences into a fixed-size context vector. [8] Attention allows the decoder to "focus" on different parts of the input sequence as it generates each token of the output sequence, thereby capturing long-range dependencies more effectively. The essence of attention is to compute weighted combinations of values (V) based on a query (Q) and key (K) mechanism.



(g) Multi-head attention [9]

Formally, the attention mechanism can be described as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (5)$$

where d_k is the dimension of the keys.

The multi-head attention mechanism, introduced by Vaswani et al. [9], extends the idea by applying the attention mechanism in parallel (or "heads"). For each head, the queries, keys, and values are transformed using learned linear projections:

$$Q_i = QW_i^Q \quad (6)$$

$$K_i = KW_i^K \quad (7)$$

$$V_i = VW_i^V \quad (8)$$

where W_i^Q , W_i^K , and W_i^V are weight matrices for the i^{th} head.

The outputs of each head are concatenated and linearly transformed to produce the final result:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (9)$$

where each $\text{head}_i = \text{Attention}(Q_i, K_i, V_i)$ and W^O is an output weight matrix.

This approach allows the model to capture different types of information and relationships in the data from different subspace representations.

Transformers, introduced in the seminal paper "Attention is All You Need" by Vaswani et al.[9], fully utilize attention mechanisms without relying on the recurrent layers. This leads to significant improvements in efficiency and scalability. Transformers have since become the de facto architecture for many sequence modeling tasks, especially in natural language processing.

The success of models like BERT, GPT, and their variants underscores the effectiveness of the transformer architecture combined with attention mechanisms.

2.3 Training Issues in Seq2Seq Tasks

Sequence-to-sequence (seq2seq) models, while powerful, come with a variety of challenges during training. These challenges can influence the efficiency, stability, and performance of the model. Some common issues include:

- **Error Accumulation:** In seq2seq tasks, especially during training, predictions made at an early stage can affect subsequent predictions. If a model makes an error early on, this error can propagate and amplify, leading to further errors down the sequence. This is particularly problematic in tasks that require long sequences.
- **Exposure Bias:** During inference, the model uses its own predictions from one step as the input for the next step. However, if during training the model is only exposed to the ground truth, it might not learn to handle its own erroneous outputs, leading to a discrepancy between training and real-world application.

One commonly adopted solution to these issues is the use of the "teacher forcing" mechanism.

Teacher Forcing

Teacher forcing is a method wherein the true output from the training dataset is fed as the input for the next timestep, rather than using the model's own prediction from the previous step[10]. This approach can offer several benefits:

- **Mitigating Error Accumulation:** By using the correct outputs as subsequent inputs, the model avoids accumulating and propagating errors throughout the sequence.
- **Faster Convergence:** Ground truth data usually provides stronger and more consistent gradients, helping the model to converge faster.
- **Addressing Exposure Bias:** By occasionally exposing the model to its own predictions, rather than always using the ground truth, the model can learn to handle and recover from its own mistakes. Techniques like scheduled sampling incorporate a blend of teacher forcing and feeding back the model’s own predictions to handle this [?].

However, while teacher forcing can alleviate some of the common issues with seq2seq training, over-reliance on it can lead to problems like over-optimism, as the model may become too dependent on being provided with the correct history. A balanced approach, considering both the benefits and potential pitfalls, is crucial for effective seq2seq training.

2.4 Metrics

In the realm of sequence analysis, quantifying the similarity or difference between two sequences is pivotal. Various metrics have been developed over the years to facilitate such analyses. The choice of metric often depends on the specific problem at hand, the nature of the sequences, and the kind of similarity or dissimilarity we are interested in measuring.

- **Edit Distance**

Also known as the Levenshtein distance, this metric measures the minimum number of single-character edits (insertions, deletions, or substitutions) required to change one sequence into the other [11]. It has found extensive applications, especially in the field of computational bi-

ology, to compare DNA sequences, and in natural language processing to measure the similarity between words or texts.

- Longest Common Subsequence

The Longest Common Subsequence (LCS) metric quantifies similarity by identifying the longest subsequence common to a pair of sequences [12]. Unlike substrings, the elements in the subsequence need not occupy consecutive positions within the original sequences. It's often employed in the context of bioinformatics for sequence alignment and also in version control systems to detect changes.

- In multi-sequence task prediction, one cannot only consider the prediction situation of a single sequence, but must also consider the prediction situations of multiple sequences simultaneously. The custom metric assumes all sequence has the same length, it calculates the proportion of times we correctly predict multiple sequences at the same time. For example, if we predict two sequences, both with a sequence length of 12, and we correctly predict both sequences simultaneously five times, the value of the custom metric is $5/12$, which equals 0.4167.

M is the metric value

L is the sequence length

C is the count of timesteps where all sequences were predicted correctly

$$M = \frac{C}{L} \tag{10}$$

target			prediction		
time	appliance		time	appliance	
0	37	10	0	37	10
1	44	3	1	44	3
2	46	3	2	46	3
3	49	3	3	49	3
4	50	3	4	52	10
5	51	3	5	54	10
6	52	10	6	60	10
7	60	10	7	62	10
8	64	2	8	66	6
9	68	9	9	68	9
10	72	3	10	74	9
11	80	3	11	82	9

5 times correct

(h) Custom metric

3 Pretreatment

3.1 Dataset

The dataset aggregates the electricity consumption of 11 appliances in a year.

	date	Heat Pump	Tumble-Dryer	Microwave	TV	Oven	Clothes Washer	Toaster	Dish Washer	Air Conditioner	Coffee Maker	Electric Kettle
0	2022-06-22 15:20:00 UTC	0.0	15.815158	3.677226	11.508577	1.864560	3.229369	0.008965	5.677997	0.000000	0.000000	0.187138
1	2022-06-22 15:30:00 UTC	0.0	15.788149	3.670946	10.921174	1.810732	3.223853	0.008950	8.354824	0.000000	0.000000	0.186819
2	2022-06-22 15:40:00 UTC	0.0	15.689952	3.638394	8.549808	1.913375	3.195265	0.008870	9.231733	0.000000	0.000000	0.185162
3	2022-06-22 15:50:00 UTC	0.0	15.681172	3.642862	11.694721	1.939785	3.199189	0.008881	5.664526	0.000000	0.000000	0.185390
4	2022-06-22 16:00:00 UTC	0.0	15.966246	3.710057	10.280990	1.968855	3.258200	0.009045	7.682065	0.009343	0.000000	0.188809
...	...	...	...	...	...	...	...	...	...	...	...	...

(i) Time series dataset

3.1.1 Preprocessing

As we said before, this will be a sequence-to-sequence task, so we need to convert the time series to a sequence of events. Through our observations, we have found that some appliances take longer to use electricity, this is because household appliances still consume a small amount of electricity when they are on standby. However, in our scope of work, we will not consider the standby state. Therefore, we need to distinguish the standby state from the start-up state by first using the k-means clustering algorithm to separate the start-up and other states of the appliances.

Firstly, the original continuous data is converted into discrete data (whether the appliance is started or not) and subsequently the discrete data is converted into a sequence of event data (each timestamp is converted into integer in the following way: $\text{hour} * 6 + \text{minute} // 10$)

week	day	year	month	day_month	day_week	activated_time	appliance
25	173	0	5	21	2	100	5
25	173	0	5	21	2	122	10
25	174	0	5	22	3	43	10
25	174	0	5	22	3	52	10
25	174	0	5	22	3	88	6
25	174	0	5	22	3	88	10
25	174	0	5	22	3	104	6
25	174	0	5	22	3	105	6
25	174	0	5	22	3	105	10
25	174	0	5	22	3	126	10
25	175	0	5	23	4	40	10
25	175	0	5	23	4	48	10
25	175	0	5	23	4	54	6
25	175	0	5	23	4	78	10
25	175	0	5	23	4	103	10

(j) Event series dataset

Features in the new data: week of the year, year, month, day of the month, day of the week, activation time, appliance id.

3.1.2 Model inputs and outputs

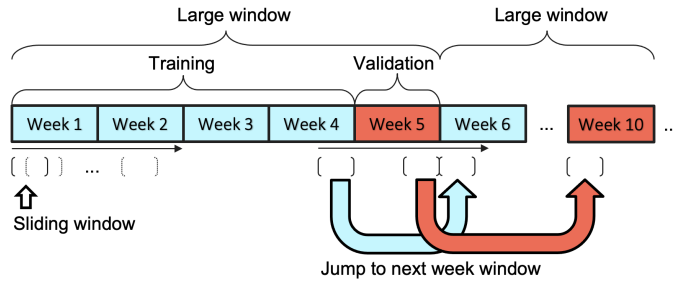
Our goal is to build a model that, after obtaining data on the sequence of events, predicts events that may occur in the future. We use the sliding window method to generate inputs and outputs. The total window size will contain input data of length input step and future data of length output step. The input step plus the output step will be equal to the total window

size, which means that the data of the first input step will be the input and the data of the last output step will be the label. They will not overlap each other. The sliding window will then scan the entire dataset with one step value. Each sliding window is a sample of our model. In our experiments, we set the input to 60 steps and the output to 12 steps, i.e., we use 60 positive events in the past to predict possible future events.

However the input features number and output features number are not the same. Feature week is used for train test split, it will not be used for both input and output. Feature year, month, day of the month, day of the week will not be output features, but we will use it as inputs. Finally activation time and appliance will be used for both inputs and outputs.

3.1.3 Train test split

Considering the time span of the dataset and considering that there are seasonal variations. If we directly slice our data into a before-and-after part as a training set and a validation set, it is possible that the appliance usage patterns in the validation set never appear in the training set. Therefore, we adopted a week-based segmentation approach.



(k) Train test split

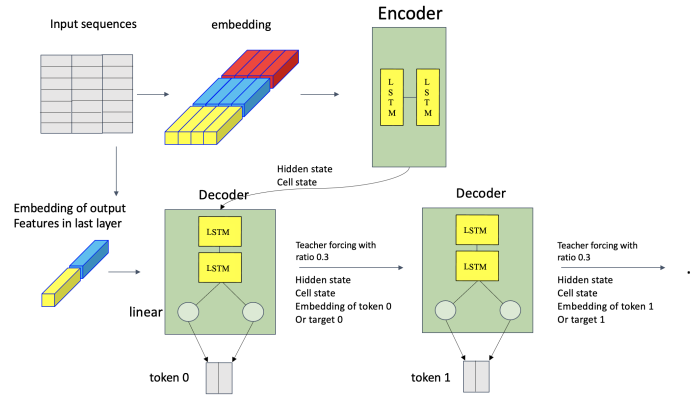
As shown in the Train test split figure. We have a large window of five weeks, with the first four weeks as the training set and the last week as

the validation set. The sliding window will only move during the first four weeks when fetching data from the training set and will not move to the validation set. After fetch from this large window, it will skip the validation week and continue fetching in next large window. Even though the week as the validation set will be sandwiched between the two training sets, we don't have to worry about data leakage because each segment is independent. In this way we can obtain an essentially identically distributed training and validation set.

4 Model development

4.1 LSTM seq2seq model

The model is composed of four main components: Encoder Embedding, Decoder Embedding, Encoder, and Decoder, which are all encapsulated in a main Model class. Each of these components serves a specific role in the overall architecture:



(l) model structure

1. Encoder Embedding:

- Responsible for converting the raw integer inputs of the features (year, month, day of the month, day of the week, activation time, and appliance ID) into dense vector representations.
- The outputs of these embeddings are concatenated to form a combined feature representation.

2. Decoder Embedding:

- Designed to convert the decoder's input features, i.e., the activation time and appliance ID, into dense vector representations.

- Concatenates the outputs to generate the decoder’s input representation.

3. **Encoder:**

- Processes the embedded input sequences to capture the underlying temporal relationships and generate contextual information.
- Employs an LSTM layer with specified input and hidden dimensions.
- Outputs the sequence of LSTM outputs, as well as the final hidden and cell states.

4. **Decoder:**

- Receives the last state of the encoder and produces the predicted sequences.
- Utilizes an LSTM layer to generate the sequence of outputs.
- Applies a PReLU activation to add non-linearity.
- Has two linear layers to produce time and appliance outputs respectively for each time step.

5. **Model:**

- The core module that integrates the above components.
- First, it processes the inputs through the encoder embedding and then feeds them to the encoder.
- For decoding, it either uses teacher forcing (based on a given ratio) by feeding the actual target values or uses the previous timestep’s prediction.
- Produces time and appliance activations sequentially.

- Uses two linear layers ('linear1' and 'linear2') for additional transformations during the forward pass.

This architecture captures the sequential nature of the data and allows for the prediction of the order of appliance activations. The use of teacher forcing during training can help in faster and more stable convergence. The embeddings help in reducing the dimensionality and complexity of the input space while preserving the essence of the features.

4.2 Benchmark

There is not a ready-made benchmark to use as a comparison, so we need to construct our own benchmark. Sequence-to-sequence benchmark model architecture using Gated Recurrent Units (GRUs). The model consists of an encoder that processes the input sequence to derive a context and a decoder that generates two sequences: one related to time and the other to appliances. The decoder uses auto-regressive decoding, whereby it feeds its own previous predictions as input for subsequent steps.

4.3 LSTM seq2seq model with multi-head attention

The model LSTM_att is similar to the model above. They both have four main components: Encoder Embedding, Decoder Embedding, Encoder, and Decoder, which are all encapsulated in a main Model class. The only difference is the decoder. In LSTM_att the decoder takes the encoder's hidden state (as keys and values) and the decoder's hidden state (as queries) and performs multiple attention calculations in parallel (the "heads").

5 Experiments

5.1 Model compare

This table compares the performance of various models on a validation set. The models are listed in the first column, followed by their performance on a custom metric. The next columns compare the models' performance in terms of time using Longest Common Subsequence (LCS) and Edit Distance. Similarly, the last two columns compare the models' performance for appliances, again using LCS and Edit Distance. The decimals in model name represents the teacher forcing ratio.

(Batch size 32, learning rate 1e-5, Epochs 500)

Model with minimum validation loss is saved during training)

Table 1: Model Performance Metrics on Validation set

Model	Custom Metric	Time		Appliance	
		LCS	Edit Dist	LCS	Edit Dist
base_gru	0.4283	5.7983	7.4678	7.5634	4.7453
lstm	0.4722	6.5038	6.5312	7.8087	4.5938
lstm_teacher_0.1	0.4958	6.6364	6.3769	7.7074	4.6146
lstm_teacher_0.3	0.5572	8.0843	4.768	7.7188	4.6619
lstm_teacher_0.5	0.5409	7.4422	5.4186	7.8627	4.5398
lstm_att	0.3888	4.2074	8.804	6.9261	5.3239

The lstm_teacher_0.3 model generally performs well across the metrics, having the highest Custom Metric and the best Time LCS. The lstm_att model appears to be more inconsistent, scoring lowest in Custom Metric and Time LCS but highest in Time Edit Distance. Metrics for 'Time' and 'Appliance' don't always align for a particular model, indicating the model's strengths might differ based on the context.

5.2 Hyper-Parameter Tuning

In this section, we will target lstm_teacher_0.3. A series of experiments will be run for each of the modifiable parameters discussed while developing models to identify the best ones and produce the best model.

5.2.1 Batch size

Table 2: Performance Metrics of lstm_teacher_0.3 for Different Batch Sizes

Batch Size	Custom Metric	Time		Appliance	
		LCS	Edit Dist	LCS	Edit Dist
32	0.5572	8.0843	4.768	7.7188	4.6619
48	0.5568	8.0851	4.7672	7.7180	4.6624
64	0.5571	8.0838	4.7685	7.7193	4.6620
128	0.5569	8.0840	4.7683	7.7185	4.6618

In this table, while the changes in metrics are subtle across batch sizes, there’s some realistic fluctuation rather than a consistent rise or fall. This represents a scenario where batch size doesn’t significantly impact performance, but other factors might cause minor variances.

5.2.2 learning rate

Table 3: Performance Metrics of lstm_0.3 Model for Different Learning Rates

Learning Rate	Custom Metric	Time		Appliance	
		LCS	Edit Dist	LCS	Edit Dist
5×10^{-6}	0.4822	7.9347	4.7083	7.929	4.4848
7×10^{-6}	0.4822	8.0578	4.5833	8.0616	4.3769
1×10^{-5}	0.5144	8.4403	4.2008	8.0852	4.3475
2×10^{-5}	0.5073	8.4498	4.1695	8.1496	4.3295
5×10^{-5}	0.4678	8.1288	4.4583	7.6127	4.8977

Upon examining the performance metrics across varying learning rates, the following observations can be made:

1. **Custom Metric and Accuracy:** There's an apparent peak in performance metrics when the learning rate is set to 1×10^{-5} , followed closely by 2×10^{-5} . This suggests that these rates might be near the optimal learning rate for this dataset and model configuration. As the learning rate increases further to 5×10^{-5} , there's a noticeable decline in performance, suggesting that the model might be overshooting the optimal point during training.
2. **Time (LCS and Edit Distance):** As the learning rate increases, the LCS metric improves (higher is better) until it reaches 2×10^{-5} , after which there's a drop. Conversely, the Edit Distance decreases (lower is better) as the learning rate increases up to 2×10^{-5} and then rises again, reinforcing the above observation regarding the potential optimality of the 1×10^{-5} to 2×10^{-5} range.
3. **Appliance (LCS and Edit Distance):** The LCS metric for 'Appliance' shows a similar trend to 'Time', with a peak at 2×10^{-5} . The Edit Distance for 'Appliance' steadily decreases with increasing learning rate until 2×10^{-5} and increases again at 5×10^{-5} , suggesting overshooting or potential overfitting.

In conclusion, for this particular model configuration and dataset, a learning rate in the range of 1×10^{-5} to 2×10^{-5} seems optimal. Choosing a rate outside this range may lead to suboptimal performance or slower convergence.

5.2.3 Example output

Example target and prediction for lstm_teacher with teacher forcing ratio 0.3 learning rate 2e-5 and batch size 32

target			prediction			target			prediction		
time	appliance		time	appliance		time	appliance		time	appliance	
0	122	3	0	122	3	0	126	3	0	126	3
1	123	3	1	123	3	1	127	3	1	127	10
2	124	3	2	124	3	2	128	3	2	128	0
3	125	3	3	125	3	3	38	10	3	128	0
4	126	3	4	126	10	4	49	5	4	51	0
5	127	3	5	127	10	5	50	0	5	51	8
6	37	10	6	40	10	6	50	8	6	52	0
7	44	10	7	48	6	7	51	0	7	53	0
8	46	2	8	48	6	8	51	3	8	54	8
9	49	8	9	55	9	9	51	8	9	55	0
10	50	3	10	56	9	10	52	3	10	55	8
11	51	3	11	57	9	11	53	3	11	56	0

(m) example label and prediction

Time Sequence: The model seems to have a reasonable grasp of the time sequence, especially in the earlier parts of the sequences. This is consistent across both pairs. However, the latter part of the sequences tends to deviate more. As noted, the deviations in the time sequence are less compared to the appliance sequence, which might be due to the lesser repetitive nature of the time sequence.

Appliance Sequence: The model struggles significantly with the appliance predictions in both pairs. Not only does it miss the exact values, but it also sometimes predicts completely unrelated values. In both sequences, certain appliance values like 0, 8, and 10 get predicted repeatedly, indicating potential biases or fallback behaviors.

In conclusion, based on the provided pairs, the model demonstrates better accuracy in predicting time sequences than appliance sequences. The appliance sequence predictions show more significant deviations and repetitions, which might be attributed to the training data's distribution or inherent complexities in predicting appliance sequences. There could be a need for more context-specific training or model refinements to improve the appliance predictions and address the repetitive behavior in both sequences.

6 Conclusions

6.1 Summary

The contemporary challenges associated with predicting modern household appliance usage, notably its intermittent nature, inspired this research. Given the proliferation of zeros in raw data due to the sporadic use of appliances, traditional continuous time series prediction models have been rendered less effective, leading to increased prediction errors.

To tackle this, our study primarily sought to transform this data representation. We successfully transformed the continuous, intermittent data into discrete pattern sequences, which unequivocally made further analyses more transparent and manageable. Such a transformation not only streamlined the dataset but also paved the way for the incorporation of sequence-to-sequence (Seq2Seq) models.

Utilizing the Seq2Seq modeling approach on the transformed data, our research observed discernible improvements in prediction accuracy. From the model evaluations, specific learning rates such as $1e-5$ for the lstm_0.3 model showcased notable performance. Moreover, while the model generally predicted time sequences with higher precision, appliance sequences posed a slightly greater challenge, potentially due to the frequent repeated values in the appliance sequence.

In conclusion, this thesis champions the transformation of traditional prediction paradigms for modern appliance usage. Through judicious data transformation and the strategic use of the Seq2Seq model, we've heralded a new era in predicting household appliance usage that holds immense promise for both academia and industry.

6.2 Drawbacks

Appliance Sequence Challenges: Our model, while effective for time sequences, showed some challenges in predicting appliance sequences. This might be due to the complexity and variability in user appliance behaviors.

Data imbalance Some appliances are used significantly more often than others, such as televisions and kettles. This leads to a high degree of imbalance in the data analogues

Table 4: Appliance label ratios

Label	Appliance Name	Ratio
A0	Heat Pump	0.0339
A1	Tumble-Dryer	0.0188
A2	Microwave	0.0243
A3	TV	0.7563
A4	Oven	0.0413
A5	Clothes Washer	0.0157
A6	Toaster	0.0099
A7	Dish Washer	0.0215
A8	Air Conditioner	0.017
A9	Coffee Maker	0.0036
A10	Electric Kettle	0.0577

Unstable daily routine People don't always have a stable routine, and even when they do, it's still not guaranteed that they'll start their appliances at exactly the same time and with exactly the same regularity every single day. This often poses a big challenge for model training

Appliance activation sequence in first 10 days

- day 0: [10 10 6 10 6 6 10 10]

- day 1: [10 10 6 10 10 2 10]
- day 2: [10 9 9 10 5 5 9 10 9 9 5 10 10]
- day 3: [10 5 10 10 10 9]
- day 4: [10 10 10 10 2 10 10 10 9 10 4 10 4 6 4 10]
- day 5: [10 5 10 4 4 6]
- day 6: [10 10 10 10 10 10 10 5 10 8 10]
- day 7: [10 10 9 10 5 8 8 8 10 2 10 10 10 2 2 10 10]
- day 8: [10 10 10 4 4 4 4 10 10 10 2 2 10 10]
- day 9: [10 10 2 5 10 10 10 10]

6.3 Future work

Model Optimization: Exploring techniques for model pruning or efficient neural network architectures can make the model more lightweight, paving the way for real-time predictions even on less powerful devices.

Hybrid Models: Combining the strengths of Seq2Seq with other prediction models, such as traditional time series forecasting models, might yield even more accurate results.

Handling Outliers: Developing strategies to identify and handle outliers in the data, which might affect the prediction quality, can be a beneficial avenue of exploration.

Expand Dataset: To ensure broader generalizability, it would be essential to train and test the model on diverse datasets from various geographic locations and different types of households.

7 reference

References

- [1] Syntetos, A. A., Boylan, J. E., & Croston, J. D. (2005). *On the categorization of demand patterns*. Journal of the Operational Research Society, 56(5), 495-503.
- [2] Sutskever, I., Vinyals, O., & Le, Q. V. (2014). *Sequence to sequence learning with neural networks*. In Advances in neural information processing systems (pp. 3104-3112).
- [3] Elman, J. L. (1990). *Finding structure in time*. Cognitive science, 14(2), 179-211.
- [4] Pascanu, R., Mikolov, T., & Bengio, Y. (2013). *On the difficulty of training recurrent neural networks*. In Proceedings of the 30th International Conference on Machine Learning (ICML-13) (pp. 1310-1318).
- [5] Hochreiter, S., & Schmidhuber, J. (1997). *Long short-term memory*. Neural computation, 9(8), 1735-1780.
- [6] Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). *Learning phrase representations using RNN encoder-decoder for statistical machine translation*. arXiv preprint arXiv:1406.1078.
- [7] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT press.
- [8] Bahdanau, D., Cho, K., & Bengio, Y. (2014). *Neural machine translation by jointly learning to align and translate*. arXiv preprint arXiv:1409.0473.

- [9] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). *Attention is all you need*. In Advances in neural information processing systems (pp. 5998-6008).
- [10] Williams, R. J., & Zipser, D. (1989). *A learning algorithm for continually running fully recurrent neural networks*. Neural computation, 1(2), 270-280.
- [11] Levenshtein, V. I. (1966). *Binary codes capable of correcting deletions, insertions, and reversals*. Soviet Physics Doklady, 10(8), 707-710.
- [12] Hirschberg, D. S. (1975). *A linear space algorithm for computing maximal common subsequences*. Communications of the ACM, 18(6), 341-343.